

DEEP LEARNING LAB

Assignment 2: MLP Classifier on Iris Dataset

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Classification

1. Aim

To implement a Multi-Layer Perceptron (MLP) neural network for multi-class classification using the Iris dataset, evaluate its performance using accuracy, confusion matrix and classification report, and use the trained model to predict the class of a new flower.

2. Objectives

- Load the Iris dataset using scikit-learn.
- Split the dataset into training and testing sets.
- Standardize the input features using StandardScaler.
- Build an MLP classifier with two hidden layers using ReLU activation.
- Train the model using the Adam optimizer.
- Evaluate the model using accuracy, confusion matrix and classification report.
- Predict the class of a new flower using the trained model.

3. Dataset Description

The notebook uses the Iris dataset available through `sklearn.datasets.load_iris()`. The dataset contains 150 samples, 4 input features and 3 target classes.

Property	Value
Number of samples	150
Number of features	4
Classes	3
Class names	setosa, versicolor, virginica
Training samples	120 (80%)
Testing samples	30 (20%)

4. Input Features

- Sepal length
- Sepal width
- Petal length
- Petal width

5. Methodology

The notebook follows a sequence of dataset loading, train-test splitting, feature scaling, MLP configuration, model training, evaluation, visualization and prediction.

Dataset Loading: The Iris dataset is loaded with `load_iris()`. `X` contains the four features and `y` contains the class labels.

Train-Test Split: `train_test_split()` is used with `test_size=0.20`, `random_state=42` and `stratify=y`, producing 120 training and 30 testing samples.

Feature Scaling: `StandardScaler` is fitted on `X_train` and then applied to both `X_train` and `X_test`.

MLP Construction: `MLPClassifier` is configured with two hidden layers of 10 neurons each, ReLU activation, Adam solver and `max_iter=1000`.

Training: The model is trained using `mlp.fit(X_train, y_train)`.

Evaluation: Predictions are generated for `X_test` and evaluated using `accuracy_score`, `confusion_matrix` and `classification_report`.

New Sample Prediction: A flower with measurements `[5.1, 3.5, 1.4, 0.2]` is scaled using the fitted scaler and classified by the trained MLP.

6. MLP Architecture

Layer	Configuration	Purpose
Input layer	4 features	Receives the four Iris measurements
Hidden layer 1	10 neurons, ReLU	Learns nonlinear feature representations
Hidden layer 2	10 neurons, ReLU	Learns higher-level representations
Output layer	3 classes	Produces the predicted Iris class

7. Important Code Used

Importing Libraries

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay,
classification_report
```

Loading Dataset

```
iris = load_iris()
X = iris.data
y = iris.target
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Feature Scaling

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

MLP Model

```
mlp = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
```

Training and Evaluation

```
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

8. Evaluation Metrics

The notebook evaluates the trained classifier using the following measures:

- Accuracy: proportion of test samples classified correctly.
- Confusion Matrix: shows the number of correct and incorrect predictions for each class.
- Precision: measures how many samples predicted as a class actually belong to that class.
- Recall: measures how many samples belonging to a class are correctly identified.
- F1-score: harmonic mean of precision and recall.

9. Confusion Matrix and Classification Report

The notebook generates a confusion matrix using `confusion_matrix()` and displays it with `ConfusionMatrixDisplay`. It also prints a classification report containing precision, recall, F1-score and support for setosa, versicolor and virginica.

The exact numerical evaluation values are execution-dependent and should be taken from the output produced when the uploaded notebook is run. The notebook itself does not contain saved output values for the accuracy or classification report.

10. Prediction of a New Flower

The notebook tests the trained model on a new flower having the measurements [5.1, 3.5, 1.4, 0.2]. The sample is first transformed with the same `StandardScaler` used during training and is then passed to the MLP classifier.

Prediction code:

```
new_flower = np.array([[5.1, 3.5, 1.4, 0.2]])
new_flower_scaled = scaler.transform(new_flower)
```

```
prediction = mlp.predict(new_flower_scaled)
print("Predicted class:", iris.target_names[prediction[0]])
```

11. Result and Observations

- The Iris dataset is successfully loaded with 150 samples and 4 features.
- The stratified 80:20 split creates 120 training samples and 30 testing samples while preserving class proportions.
- Feature standardization is applied before training the MLP.
- The classifier contains two hidden layers with 10 neurons each and uses ReLU activation.
- The Adam optimization algorithm is used for training.
- The model produces predictions for the test set and for the supplied new flower.
- The notebook provides a confusion matrix and classification report for detailed performance analysis.

12. Advantages of MLP

- Can learn nonlinear relationships between input features and target classes.
- Can be applied to multi-class classification problems.
- The implementation through scikit-learn is concise and easy to reproduce.
- Feature scaling can improve neural-network training behavior.

13. Limitations

- The Iris dataset is small and simple, so it does not represent the complexity of large real-world datasets.
- The selected architecture and hyperparameters may not be optimal for every dataset.
- MLP performance can depend on feature scaling, network size and training settings.

14. Conclusion

A Multi-Layer Perceptron classifier was implemented successfully on the Iris dataset using scikit-learn. The workflow included dataset preparation, stratified train-test splitting, standardization, construction of a two-hidden-layer MLP, training with Adam, and evaluation using accuracy, confusion matrix and classification report. The trained model was also used to classify a new flower sample.

15. Viva Questions and Answers

Q: What is an MLP?

A: A Multi-Layer Perceptron is a feed-forward neural network containing an input layer, one or more hidden layers and an output layer.

Q: Why is StandardScaler used?

A: It standardizes features so they have approximately zero mean and unit variance, which can help neural-network optimization.

Q: Why is ReLU used?

A: ReLU introduces nonlinearity while being computationally simple and generally helping deep networks train effectively.

Q: What is the role of Adam?

A: Adam is an adaptive optimization algorithm used to update model parameters during training.

Q: Why is stratify=y used?

A: It preserves the class distribution when creating the training and testing subsets.

Q: What does a confusion matrix show?

A: It shows correct and incorrect predictions broken down by actual and predicted class.

Q: Why is the test set scaled using transform rather than fit_transform?

A: The scaler must be fitted only on training data to avoid using information from the test set during preprocessing.

16. Source Notebook

This report is based directly on the uploaded Assignment_2.ipynb notebook. Its implementation uses NumPy, Matplotlib and scikit-learn's Iris dataset and MLPClassifier.